

PROPOSTA DE DOMÍNIO - RepFinder API

Plataforma de Busca e Gestão de Oportunidades de Emprego

1. Descrição do Domínio

RepFinder é uma plataforma distribuída de busca e gestão de oportunidades de emprego que conecta **candidatos (estudantes)** com **representantes de empresas (recrutadores)** de forma assíncrona e escalável.

O sistema permite que estudantes busquem e se candidatem a vagas de emprego publicadas por representantes de empresas, enquanto estes gerenciam o ciclo completo de recrutamento: criação de vagas, visualização de candidaturas, aprovação/rejeição de aplicações e acompanhamento de status.

2. Justificativa

- **Relevância:** Mercado de recrutamento crescente com necessidade de plataformas ágeis e assíncronas
 - **Arquitetura Orientada a Eventos:** Suporta notificações em tempo real para novos candidatos sem polling contínuo
 - **Escalabilidade:** Backend em Node.js + MOM permite lidar com múltiplas requisições simultâneas
 - **Dois Perfis Distintos:** Diferenciação clara entre candidato (consumidor) e representante (produtor), conforme exigência do projeto
-

3. Perfis de Usuário

3.1 Perfil Cliente - Student (Candidato)

- Registra conta com email, nome e senha
- Autentica-se para acessar funcionalidades
- Visualiza lista de vagas disponíveis
- Consulta detalhes de uma vaga
- Aplica para vagas (máximo 1 aplicação por vaga)
- Acompanha status de suas candidaturas (pending, accepted, rejected)
- Recebe notificações quando representante aprova/rejeita sua aplicação

3.2 Perfil Prestador - Representative (Recrutador)

- Registra conta com email, nome, senha e role='representative'

- Autentica-se e acessa painel de recrutamento
 - Cria e publica novas vagas de emprego
 - Visualiza vagas que criou
 - Edita/deleta suas vagas
 - Visualiza todas as candidaturas recebidas para uma vaga
 - Aprova ou rejeita candidatos
 - Recebe notificações de novas aplicações assincronamente
-

4. Principais Funcionalidades

Fluxo de Autenticação

1. Novo usuário se registra (email, nome, senha, role)
2. Sistema valida email único
3. Gera token JWT com validade de 7 dias
4. Usuário faz login com credenciais
5. Sistema valida password com PBKDF2-SHA512
6. Retorna token de acesso

Fluxo de Vagas (Representante)

1. Representante cria vaga (title, description)
2. Sistema atribui automaticamente providerId (ID do representante)
3. Sistema publica evento “vacancy.created” no MOM
4. Vaga fica visível para candidatos

Fluxo de Candidaturas (Candidato)

1. Candidato visualiza lista de vagas públicas
2. Seleciona uma vaga e aplica
3. Sistema verifica se já não existe aplicação do mesmo candidato
4. Cria aplicação com status=‘pending’
5. Sistema publica evento “application.created” no MOM
6. Representante é notificado assincronamente

Fluxo de Aprovação (Representante)

1. Representante visualiza candidaturas de suas vagas
 2. Seleciona candidata e aprova/rejeita
 3. Sistema atualiza status (pending → accepted|rejected)
 4. Sistema publica evento “application.status_updated”
 5. Candidato é notificado assincronamente do resultado
-

5. Regras de Negócio

Regra	Descrição
Email Único	Email deve ser único no cadastro de usuários
Senha Mínima	Senha com mínimo 6 caracteres
Role Obrigatório	Usuário deve ser 'student' ou 'representative'
Vaga = Representative	Apenas representantes podem criar vagas
Ownership Vaga	Apenas proprietário pode editar/deletar vaga
1 App por Vaga	Candidato só pode aplicar uma vez por vaga
Status Workflow	Aplicação: pending → (accepted rejected); não pode reverter
App = Criador	Apenas criador da aplicação pode deletá-la
App Pending	Aplicação só pode ser deletada em status pending
Cascade Vaga	Vaga com aplicações não pode ser deletada
Ownership Approval	Apenas dono da vaga pode aprovar/rejeitar aplicações
JWT 7 dias	Token expira em 7 dias; login obrigatório após expiração

6. Estrutura Técnica (Sprint 1)

Backend REST

- **Framework:** Hono 4.12.16 (Node.js lightweight)
- **Linguagem:** TypeScript com strict mode
- **Banco:** MariaDB via mysql2/promise
- **Autenticação:** JWT HS256 + PBKDF2-SHA512
- **Validação:** DTOs com middleware customizado
- **Padrão:** Clean Architecture (routes → service → repo)

Endpoints Implementados (15 total)

Auth: register, login (2)

Users: getMe, updateMe (2)

Vacancies: create, list, listMine, getById, update, delete (6)

Applications: create, listMine, listByVacancy, updateStatus, delete (5)

7. Próximas Sprints

- **Sprint 2:** Integração com MOM (RabbitMQ/Redis) para eventos assíncronos

- **Sprint 3:** App Flutter para cliente (candidato)
 - **Sprint 4:** App Flutter para prestador (recrutador) + fluxo end-to-end
-

Data de Submissão: 10/05/2026

Prazo Sprint 1: 11/05/2026